# Hotel Reservation System

Documentación completa del sistema, su flujo y los cambios por historia de usuario

Proyecto #1 — Desarrollo asistido por IA · Tópicos Especiales II

Repositorio: ShiirooS/proyecto1-dsaia (base: natanael-lima/hotel-reservation-system, rama `developer`)
Estado documentado al 28 de octubre de 2026

## Contenido


1. Visión general y alcance
2. Arquitectura
3. Modelo de datos
4. Seguridad y roles
5. Flujos del sistema
6. API: resumen de endpoints
7. Cambios implementados por historia de usuario (HU-00 a HU-05)
8. Calidad: pruebas, CI y revisiones
9. Cómo ejecutarlo
10. Estado actual y pendientes


## 1. Visión general y alcance

Sistema web de reservas de hotel. Un visitante explora hoteles y habitaciones; un usuario registrado reserva con una tarjeta (nueva o guardada), consulta y cancela sus reservas; un administrador gestiona el inventario y consulta la previsión de demanda. El proyecto parte del repositorio de código abierto de natanael-lima y se amplía con tres historias obligatorias del curso y dos extra elegidas por el equipo.

| HU | Título | Tipo | Estado |
|---|---|---|---|
| HU-00 | Base técnica (build, perfiles, CI, errores) | Habilitadora | Mergeada |
| HU-01 | Reservar con datos de pago guardados | Obligatoria | Mergeada |
| HU-02 | Calendario dinámico de disponibilidad | Obligatoria | Mergeada |
| HU-03 | Gráficos de previsión de demanda (admin) | Obligatoria | Mergeada |
| HU-04 | Control de acceso por roles | Extra 1 | Mergeada |
| HU-05 | Mis reservas y cancelación | Extra 2 | Mergeada |

## 2. Arquitectura

**Frontend (SPA)**

React 18 + Vite 5 + TypeScript + Tailwind + React Router 6. Un único cliente HTTP (axios, `services/axiosConfig.ts`) con interceptor 401/403. Vitest + Testing Library.

**Backend (API REST)**

Spring Boot 3.3.2, Java 17. Capas: `controller` → `service` / `service/imp` → `repository` → `entity`. Lógica pura en `domain/`. JWT stateless.

**Datos**

H2 en modo PostgreSQL para dev/test (con datos de ejemplo). PostgreSQL real con perfil `postgres` y `docker-compose.yml`.

**Decisiones de arquitectura (ADR):** JWT stateless (001); H2 en dev/test y PostgreSQL real (002); pasarela de pago simulada con tokenización (003); algoritmo de previsión de demanda (004); disponibilidad derivada de las reservas (005); reloj inyectable y zona horaria de check-in (006); base `developer` en lugar de `main` (007).

**Reglas transversales:** errores con un único sobre `ApiResponse{message, code, errors}`; dinero en enteros (USD); `Clock` inyectable (zona America/Panama, check-in 15:00, máximo 30 noches); un recurso ajeno responde 404, no 403.

## 3. Modelo de datos

| Entidad | Campos principales | Notas |
|---|---|---|
| User | username (email), password (BCrypt), fullName, role (USER/ADMIN), createdAt | Rol guardado como texto (`@Enumerated(STRING)`). |
| Hotel | name, location, description, rating, image, province | Catálogo público. |
| Room | hotel, roomNumber, type, pricePerNight, capacity, amenities, availbility (activa) | Se bloquea con lock pesimista al reservar. |
| Reservation | room, user, startDate, endDate (LocalDate), guests, totalAmount, status (CONFIRMED/CANCELLED), createdAt, cancelledAt | La noche d está ocupada si start ≤ d < end. |
| Payment | reservation, amount, refundedAmount, status (COMPLETED/REFUNDED…), cardBrand, cardLast4, transactionId, paymentDate | Sin PAN ni CVV. |
| PaymentMethod | user, token, brand, last4, expMonth, expYear, holderName, isDefault, createdAt | Solo referencia tokenizada. |
| Review | user, hotel, rating, comment | Solo autenticados escriben. |

## 4. Seguridad y roles

- **Autenticación:** `POST /auth/login` devuelve un JWT (claim `uid`). El secreto se lee de `app.jwt.secret` (Base64); fuera de dev/test no tiene valor por defecto.
- **Autorización:** `SecurityConfig` declara cada ruta pública de forma explícita; el resto exige autoridad `USER` o `ADMIN` (`hasAuthority`, sin prefijo ROLE_).
- **Propiedad:** tarjetas y reservas se filtran por el usuario del token; un recurso ajeno responde 404.
- **Datos sensibles:** el número completo y el CVV nunca se persisten ni aparecen en logs (probado con `SensitiveDataTest`).
- **Anónimo en ruta protegida:** 401; autenticado sin permiso: 403.

| Recurso | Público | USER | ADMIN |
|---|---|---|---|
| Catálogo: hoteles, habitaciones, disponibilidad, reseñas (GET) | ✓ | ✓ | ✓ |
| Alta/edición/baja de hoteles y habitaciones | ✗ | ✗ | ✓ |
| Reservar, tarjetas propias, mis reservas, cancelar | ✗ | ✓ (dueño) | ✓ (dueño) |
| Usuarios, pagos y reservas globales (administración) | ✗ | ✗ | ✓ |
| Previsión de demanda | ✗ | ✗ | ✓ |

# 5. Flujos del sistema

## 5.1 Acceso (login y roles)

**1. Login**
El usuario envía email y contraseña a `/auth/login`.

**2. Token**
El backend valida con BCrypt y devuelve el JWT con `uid`.

**3. Sesión**
`AuthContext` guarda token, usuario y rol (`/api/user/current`).

**4. Rutas**
`ProtectedRoute` exige rol; el panel admin solo entra ADMIN.

**Token inválido**
El interceptor recibe 401/403 y cierra la sesión.

## 5.2 Consulta de disponibilidad (calendario)

**1. Detalle**
El visitante abre una habitación.

**2. Consulta**
El hook `useAvailability` pide `get-availability/{id}?from&to` de los dos meses visibles.

**3. Cálculo**
`OccupancyCalculator` marca como ocupadas las noches de reservas CONFIRMED.

**4. Selección**
El calendario pinta libres/ocupados y valida el rango (máx. 30 noches, sin cruzar ocupadas).

**5. Book now**
Con rango válido se habilita la reserva.

## 5.3 Reservar con tarjeta guardada o nueva

**1. Pago**
El modal muestra la tarjeta predeterminada (un clic) o un formulario de tarjeta nueva con «guardar».

**2. Validación**
`POST /api/reservation/book`: formato (400), fuente de pago única, Luhn/marca/CVV/vencimiento (422), fechas y huéspedes (422).

**3. Bloqueo**
Se bloquea la fila de la habitación y se comprueba el solape.

**Conflicto**
Fechas ya reservadas → 409, no se cobra.

**4. Cobro**
Pasarela simulada (`tok_`, `txn_`). La tarjeta terminada en 0002 se rechaza → 402 y no se guarda nada.

**5. Confirmación**
Se crean Reservation + Payment en una transacción; si pidió guardar, se guarda el token. Respuesta 201 con `Location`.

## 5.4 Mis reservas y cancelación (HU-05)

**1. Listado**
`GET /api/reservation/my-reservations` devuelve solo las del usuario.

**2. Cancelar**
`PUT /api/reservation/cancel-reservation/{id}`.

**3. Política**
`CancellationPolicy`: con 48 h o más antes del check-in, reembolso total; si no, sin reembolso.

**4. Resultado**
Reserva CANCELLED, noches liberadas y Payment REFUNDED cuando corresponde. Cancelar desde el check-in → 409.

### 5.5 Previsión de demanda (HU-03)

**1. Histórico**
Ocupación mensual de 24 meses derivada de las reservas.

**2. Modelo**
`DemandForecastCalculator` regresión lineal (OLS) con estacionalidad mensual (ADR-004).

**3. Reglas**
`PricingRules` : ajuste de precio sugerido y acción de inventario.

**4. Gráficos**
El panel admin muestra KPIs, ocupación real + tendencia + previsión, estacionalidad y tabla de precio e inventario (recharts).

## 6. API: resumen de endpoints

| Método y ruta | Acceso | HU |
|---|---|---|
| `POST /auth/login`, `POST /auth/registration-user` | Público | HU-04 |
| `GET /api/user/current` | Autenticado | HU-04 |
| `GET /api/hotel/get-all`, `get-hotel/{id}` | Público | base |
| `GET /api/room/get-all?activeOnly`, `get-room/{id}` | Público | HU-02 |
| `GET /api/room/get-availability/{id}?from&to` | Público | HU-02 |
| `GET /api/payment-method/get-all`, `POST registration-payment-method`, `PUT set-default/{id}`, `DELETE delete-payment-method/{id}` | Dueño | HU-01 |
| `POST /api/reservation/book` | Autenticado | HU-01 |
| `GET /api/reservation/my-reservations/{id}` | Dueño | HU-01 |
| `GET /api/reservation/my-reservations`, `PUT cancel-reservation/{id}` | Dueño | HU-05 |
| `GET /api/analytics/demand-forecast?horizon&hotelId` | ADMIN | HU-03 |
| CRUD de hoteles, habitaciones, usuarios, pagos y reservas | ADMIN | HU-04 |

**Códigos de error frecuentes:** 400 `VALIDATION_ERROR` ; 401 sin sesión; 402 pago rechazado; 404 recurso inexistente o ajeno; 409 `ROOM_NOT_AVAILABLE` ; 422 reglas de negocio ( `CARD_INVALID`, `CARD_EXPIRED`, `MAX_NIGHTS_EXCEEDED`, `CHECKIN_IN_PAST`, `GUESTS_EXCEED_CAPACITY` …).

## 7. Cambios implementados por historia de usuario

### HU-00 — Base técnica Mergeada · PR #8

- Se eliminó `bin/` versionado y se corrigieron 37 errores de TypeScript para que `tsc -b` compile.

- Perfiles `dev` (H2), `test` y `postgres`, más `docker-compose.yml`.
- `User.role` pasó de ordinal a texto.
- Errores unificados con `ApiResponse`, `BusinessException`, `ConflictException`, `ResourceNotFoundException` y `GlobalExceptionHandler`.
- CI en GitHub Actions (backend y frontend), Vitest configurado y `ExistingApiCharacterizationTest` que fija el comportamiento previo.

### HU-01 — Reservar con datos de pago guardados Mergeada · PR #13

- **Dominio:** `CardValidator` (Luhn, marca VISA/MASTERCARD/AMEX, CVV, vencimiento, `checkFormat`) y `CancellationPolicy`, ambos sin dependencias de Spring.
- **Modelo y servicios:** `PaymentMethod`, `PaymentMethodService`, `PaymentGateway` simulada y `BookingService.book`.
- **API:** `/api/payment-method/*` y `POST /api/reservation/book` (201 + Location).
- **Concurrencia:** bloqueo pesimista de la habitación (también en las reservas del admin) y aislamiento READ_COMMITTED.
- **Frontend:** `ConfirmBookingModal` como paso de pago, `CardForm`, `PaymentMethodSelector` y `PaymentMethodsPage` (`/payment-methods`).
- **Revisión:** el agente senior-architect-backend aprobó con cambios; se corrigió la fuga de PAN/CVV en el log DEBUG de Bean Validation.
- **Pruebas:** BookingTest (15), SensitiveDataTest (2), CardValidatorTest, CancellationPolicyTest; frontend 71 tests.

### HU-02 — Calendario dinámico Mergeada · PR #11 y #12

- Disponibilidad derivada de las reservas CONFIRMED con `OccupancyCalculator` (noche ocupada si start ≤ d < end).
- `Reservation` con `LocalDate` y `ReservationStatus`; solapes → 409/422 también en las reservas del admin.
- `GET /api/room/get-availability/{id}` y `get-all?activeOnly`; datos de ejemplo de 2 hoteles, 8 habitaciones y 60 días de reservas.
- **Frontend:** `components/calendar/*` (calendario propio de dos meses), `RoomDetailsPage` con precio por noches.
- **Fix #12:** un test intermitente del CI (`RoomDetailsPage`) se corrigió desmontando antes de resetear los mocks.
- **Pruebas:** RoomAvailabilityTest y 49 tests de frontend.

### HU-03 — Previsión de demanda Mergeada · PR #16

- **Dominio:** `DemandForecastCalculator` (OLS con tendencia y estacionalidad mensual sobre 24 meses, redondeo a 4 decimales) y `PricingRules` (precio +15/+8/0/−10 % e inventario con objetivo 80 %, en BigDecimal), más `OccupancyCalculator.nightsPerMonth`.
- **API:** `GET /api/analytics/demand-forecast?horizon&hotelId` (solo ADMIN): 400 `INVALID_PARAMETER`, 404 `HOTEL_NOT_FOUND`, 422 `INSUFFICIENT_HISTORY`. Solo cuentan habitaciones activas.
- **Datos:** histórico de 24 meses en el perfil dev, con picos en julio, agosto y diciembre y tendencia de +0.5 pp/mes.
- **Frontend:** dashboard de admin con KPIs, `ForecastChart`, `SeasonalityChart`, `ForecastTable` y selectores de horizonte y hotel; `recharts` 2.15.4 cargado con `React.lazy` (el bundle de huéspedes se mantiene en 314 kB).

- **Pruebas:** vectores normativos del ADR-004 (DemandForecastCalculatorTest, PricingRulesTest), AnalyticsControllerTest, DevDataSeederTest y tests del dashboard.

### HU-04 — Control de acceso por roles Mergeada · PR #9 y #10

- `SecurityConfig` con la matriz de permisos; se eliminó el GET público global que exponía usuarios y pagos.
- JWT con secreto configurable y claim `uid`; el filtro tolera tokens inválidos o caducados.
- `CurrentUserService`, `AdminBootstrap` y datos de ejemplo (admin@hotel.com y demo@hotel.com).
- **Revisión:** el agente rechazó la primera versión por 3 críticos (mass assignment de `id`, pagos accesibles a USER, toma de cuenta por renombrado); se corrigieron con tests.
- **Frontend:** `AuthContext`, `ProtectedRoute` por rol, `AdminLoginPage`, interceptor 401/403.
- **Fix #10 (SonarCloud S7044/S8476):** `toPathId` valida los ids antes de armar URLs.

### HU-05 — Mis reservas y cancelación Mergeada · PR #15

- **API:** `GET /api/reservation/my-reservations` (200 `[]` si no hay) y `PUT /api/reservation/cancel-reservation/{id}`, en `BookingService`.
- **Política:** con 48 h o más antes del check-in → Payment `REFUNDED` con el total; menos de 48 h → cancelada sin reembolso; desde el check-in o ya cancelada → 409; reserva ajena → 404. La cancelación bloquea la reserva para que dos PUT simultáneos no reembolsen dos veces.
- **Pasarela:** `PaymentGateway.refund` en la pasarela simulada.
- **Frontend:** `/bookings` con pestañas Próximas, Pasadas y Canceladas, `ReservationCard` y `CancelReservationModal` accesible (foco atrapado, Esc).
- **Pruebas:** MyReservationsTest (10, con reloj mutable para el límite exacto de 48 h) y 15 tests de frontend.

## 8. Calidad: pruebas, CI y revisiones

| Mecanismo | Detalle |
|---|---|
| Pruebas backend | JUnit + MockMvc sobre H2 (106 tests): caracterización, acceso por roles, reservas, disponibilidad, datos sensibles y lógica de dominio (parametrizadas). |
| Pruebas frontend | Vitest + Testing Library: 96 tests (calendario, pagos, rutas protegidas, servicios). |
| CI | GitHub Actions: build y tests de backend y frontend en cada push y PR. |
| Calidad estática | SonarCloud Quality Gate en los PR. |
| Revisión | Un PR por HU, revisión (agentes especializados y revisión humana) y merge a `develop`. Los fixes van en ramas `fix/HU-XX-descripcion`. |

## 9. Cómo ejecutarlo

- **Backend (dev, H2):** `cd hotel-reservation-system-backend && ./mvnw spring-boot:run` (puerto 3000). Usuarios de ejemplo: `admin@hotel.com / Admin12345`, `demo@hotel.com / Demo12345`.
- **Backend (PostgreSQL):** `docker compose up -d` y perfil `postgres` con `app.jwt.secret` definido.
- **Frontend:** `cd hotel-reservation-system-frontend && npm ci && npm run dev`.

- **Pruebas:** `./mvnw test` y `npx vitest run`.

## 10. Estado actual y pendientes

> Las seis historias (HU-00 a HU-05) están mergeadas en `develop` y sus issues cerrados. `develop` pasa 106 tests de backend, 96 de frontend, `tsc -b` y el build.

- Deuda menor de HU-01: lock failure → 409/503, unique constraint en `payment_method`, `ReservationConcurrencyTest`, `savedPaymentMethodId` siempre presente.
- Evidencia final: bitácora de uso de IA, respuestas incorrectas, cambios rechazados, decisiones humanas y material de la presentación.

Documentos de detalle en `docs/`: `architecture/ARCHITECTURE.md`, `architecture/API-CONTRACT.md`, `architecture/adr/`, `user-stories/`, `estado/`, `reviews/`, `CONTEXTO.md` y `PENDINGS.md`.